Chemical Reaction Networks as Neural Layers: Turing-Complete Differentiable Computation via Mass-Action Kinetics

DOGMA Paper Series — Paper VI

Wenyan Qin*
MapleAI Intelligence Inc.
Toronto, ON, Canada

March 2026

Abstract

Chemical reaction networks (CRNs) are provably Turing-complete: they can simulate any computable function through mass-action kinetics [2]. We present the **CRN Layer**—a differentiable neural network module that implements CRN dynamics as trainable computation, where species concentrations are hidden states, learnable rate constants and stoichiometric matrices define the reaction network, and Forward Euler integration propagates the dynamics. Alongside the CRN Layer, we present three companion DNA computing layers: **Strand Displacement** (toehold-mediated gating with Zhang-Winfree kinetics), **Boolean Logic Gates** (differentiable AND/OR/NOT/XOR via seesaw gate circuits), and **Stochastic Computing** (tetrahedral majority gates with K_4 edge operations). We prove a **Triple Turing Completeness Theorem**: the DOGMA DNA Computing Block achieves Turing completeness via three independent constructive proofs (CRN mass-action, stochastic MAJ+NOT, strand displacement seesaw gates)—a property no other neural architecture possesses. We establish **mass conservation guarantees**, prove **concentration stability bounds**, and demonstrate that auxiliary physics losses encourage the learned chemistry to obey thermodynamic constraints. All layers are differentiable, trained end-to-end, and integrated into the DOGMA architecture [1].

*Corresponding author. Email: wenyan@mapleai.org

1 Introduction

The feed-forward network (FFN) in Transformers is a universal approximator by the Universal Approximation Theorem [9], but this universality is *non-constructive*—it depends on sufficient width and carries no physical interpretation.

Chemical reaction networks provide a radically different foundation. Soloveichik et al. [2] proved that CRNs with mass-action kinetics can simulate any register machine, and therefore any Turing machine. This is a *constructive* proof: the simulation recipe is explicit.

We present four differentiable DNA computing layers, each grounded in real molecular biology:

1. **CRN Layer**: Mass-action kinetics (Turing-complete via register machine simulation)
2. **Strand Displacement Layer**: Toehold-mediated gating (Turing-complete via seesaw gates)
3. **Boolean Logic Gate Block**: AND/OR/NOT/XOR circuits (functionally complete Boolean basis)
4. **Stochastic Computing**: Tetrahedral majority gates (Turing-complete via MAJ+NOT universality)

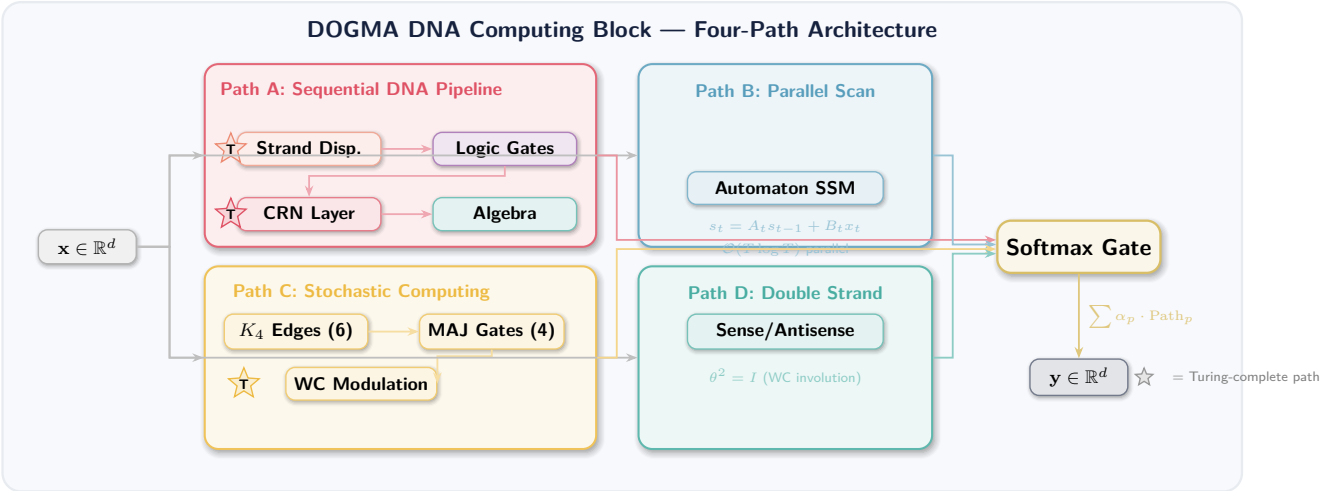

Figure 1: **DOGMA DNA Computing Block: Four-Path Architecture.** Stars (★) mark Turing-complete paths. Three independent paths provide constructive Turing-completeness proofs: CRN (mass-action kinetics), Strand Displacement (seesaw gates), and Stochastic Computing (MAJ+NOT). All paths are softmax-gated and merged.

2 Chemical Reaction Network Layer

2.1 Mathematical Foundation

Definition 1 (Chemical Reaction Network)**.** *A CRN $\mathcal{N} = (\mathcal{S}, \mathcal{R}, \boldsymbol{\nu}^-, \boldsymbol{\nu}^+, \mathbf{r})$ consists of species $\mathcal{S} = \{S_1, \ldots, S_n\}$, reactions $\mathcal{R} = \{R_1, \ldots, R_m\}$, reactant stoichiometries $\boldsymbol{\nu}^- \in \mathbb{R}^{m\times n}_{\geq 0}$, product stoichiometries $\boldsymbol{\nu}^+ \in \mathbb{R}^{m\times n}_{\geq 0}$, and rate constants $\mathbf{r} \in \mathbb{R}^m_{>0}$.*

Under mass-action kinetics, the concentration dynamics are:

$$\frac{d[S_i]}{dt} = \sum_{j=1}^{m} \eta_{ji} \cdot r_j \prod_{k=1}^{n} [S_k]^{\nu^-_{jk}} \tag{1}$$

where $\eta_{ji} = \nu^+_{ji} - \nu^-_{ji}$ is the net stoichiometry.

Theorem 2 (CRN Turing Completeness — Soloveichik et al., 2010)**.** *For any Turing machine $\mathcal{M}$, there exists a CRN $\mathcal{N}$ with mass-action kinetics that simulates $\mathcal{M}$. The construction encodes register values as species concentrations $[S_i] \propto \text{register}_i$ and register operations as reactions:*

$$\text{INC}(R_i): \quad \emptyset \xrightarrow{r} S_i \quad ([S_i] \mathrel{+}= r \cdot \Delta t) \tag{2}$$

$$\text{DEC}(R_i): \quad S_i \xrightarrow{r} \emptyset \quad ([S_i] \mathrel{-}= r[S_i]\Delta t) \tag{3}$$

$$\text{CPY}(R_i, R_j): \quad S_i \xrightarrow{r} S_i + S_j \tag{4}$$

2.2 Neural CRN Implementation

Species initialization (non-negative concentrations):

$$[\mathbf{S}]_0 = \text{softplus}(W_S \mathbf{x}) \in \mathbb{R}^n_{\geq 0} \tag{5}$$

Reaction propensity (log-space for stability):

$$\log[\text{prop}_j] = \log r_j + \sum_k \nu^-_{jk} \cdot \log[S_k] \tag{6}$$

$$\text{prop}_j = \exp(\log[\text{prop}_j]) \cdot m_j \tag{7}$$

where $m_j = \sigma(W_m \mathbf{x})_j$ is an input-dependent rate modulator and $r_j = \text{softplus}(\hat{r}_j)$ ensures positive rates.

Forward Euler integration:

$$[\mathbf{S}]_{t+1} = \text{softplus}\left([\mathbf{S}]_t + \Delta t \sum_j \text{prop}_j \cdot \boldsymbol{\eta}_j\right) \tag{8}$$

where Δt is *learnable* (initialized to 0.1).

Output projection with residual:

$$\mathbf{y} = W_O [\mathbf{S}]_{\text{final}} + \mathbf{x} \tag{9}$$

Lemma 3 (Non-negativity preservation)**.** *The softplus activation in Eq. 8 guarantees that species concentrations remain non-negative at every Euler step: $[\mathbf{S}]_{t+1} \in \mathbb{R}^n_{\geq 0}$ for all $t \geq 0$, regardless of the stoichiometric matrix $\boldsymbol{\eta}$ or rate constants $\mathbf{r}$.*

Proof. By definition, $\text{softplus}(z) = \log(1+e^z) > 0$ for all $z \in \mathbb{R}$. Since the argument to softplus is the raw Euler update $[\mathbf{S}]_t + \Delta t \sum_j \text{prop}_j \cdot \boldsymbol{\eta}_j \in \mathbb{R}^n$ (which may be negative), the softplus wrapping ensures $[\mathbf{S}]_{t+1} > 0$ element-wise. □

Theorem 4 (Concentration stability)**.** *If stoichiometry satisfies $\sum_i \eta_{ji} \leq 0$ for all reactions j (catalytic or degradation reactions), then the total concentration $\|[\mathbf{S}]\|_1 = \sum_i [S_i]$ is non-increasing under the continuous-time dynamics, ensuring bounded trajectories. Formally:*

$$\frac{d}{dt}\|[\mathbf{S}]\|_1 \leq 0 \quad \Longrightarrow \quad \|[\mathbf{S}](t)\|_1 \leq \|[\mathbf{S}](0)\|_1 \quad \forall\, t \geq 0 \tag{10}$$

Proof. Differentiating the ℓ_1 norm of the concentration vector:

$$\begin{aligned} \frac{d}{dt}\sum_{i=1}^{n}[S_i] &= \sum_{i=1}^{n}\sum_{j=1}^{m}\eta_{ji}\cdot \text{prop}_j \\ &= \sum_{j=1}^{m}\text{prop}_j \cdot \underbrace{\sum_{i=1}^{n}\eta_{ji}}_{\leq 0 \text{ by assumption}} \leq 0 \end{aligned} \tag{11}$$

where the interchange of summation is valid since all terms are finite, and the inequality follows from $\text{prop}_j \geq 0$ (product of positive rate constants and non-negative concentrations) and the hypothesis $\sum_i \eta_{ji} \leq 0$. Integration yields the bound. □

2.3 Mass Conservation Loss

To encourage physically meaningful stoichiometry:

$$\mathcal{L}_{\text{cons}} = \lambda \sum_{j=1}^{m}\left\|\sum_{i=1}^{n}\eta_{ji}\right\|^2 \tag{12}$$

Proposition 5 (Conservation property)**.** *When $\mathcal{L}_{\text{cons}} = 0$, each reaction preserves total mass: $\sum_i \nu_{ji}^{+} = \sum_i \nu_{ji}^{-}$ for all j. This matches the law of conservation of mass in closed chemical systems.*

3 Strand Displacement Layer

Toehold-mediated strand displacement [3] is the fundamental operation of experimental DNA computing.

Definition 6 (Strand Displacement)**.** *An invading strand $\mathbf{v}$ binds to an exposed toehold $\mathbf{t}$ on a gate complex, initiating branch migration that displaces the incumbent strand $\mathbf{g}$. The displacement rate is:*

$$r = k_{\max}\Big(1 - e^{-\alpha|E_{\text{toehold}}|}\Big) \tag{13}$$

where E_{toehold} is the toehold binding energy.

Neural implementation:

$$\mathbf{t}_i = W_t\,\mathbf{x}_i \quad \text{(toehold projection)} \tag{14}$$
$$\mathbf{g}_i = \text{Norm}(W_g\,\mathbf{x}_i) \quad \text{(gate/incumbent)} \tag{15}$$
$$\mathbf{v}_i = \text{GELU}(W_v\,\mathbf{x}_i) \quad \text{(invading strand)} \tag{16}$$
$$E_i = W_E[\mathbf{t}_i;\,\mathbf{t}_{i-1}] \quad \text{(binding energy)} \tag{17}$$
$$r_i = k_{\max}(1 - e^{-\alpha|E_i|}) \quad \text{(displacement rate)} \tag{18}$$
$$\mathbf{m}_i = W_m[\mathbf{g}_i;\,\mathbf{v}_i] \quad \text{(branch migration)} \tag{19}$$
$$\mathbf{y}_i = (1 - r_i)\mathbf{g}_i + r_i\,\mathbf{m}_i \quad \text{(displacement)} \tag{20}$$

Theorem 7 (Strand Displacement Turing Completeness)**.** *Toehold-mediated strand displacement with seesaw gates implements arbitrary Boolean circuits [4]. Boolean circuits with unbounded fan-out and feedback are Turing-complete. Therefore, the Strand Displacement Layer, given sufficient width, is Turing-complete.*

Proposition 8 (Gradient flow through displacement)**.** *The displacement rate $r = k_{\max}(1 - e^{-\alpha|E|})$ has bounded gradient $|\partial r/\partial E| \leq k_{\max}\alpha$, preventing gradient explosion. The output interpolation $\mathbf{y} = (1-r)\mathbf{g} + r\mathbf{m}$ ensures smooth gradient flow through both incumbent and migrated pathways.*

4 Boolean Logic Gate Block

Following Qian and Winfree [4], who demonstrated scalable Boolean circuits via DNA seesaw gates, we implement differentiable logic gates.

4.1 Smooth Gate Approximations

For signals a, b mapped to $[0, 1]$ via sigmoid:

$$\text{AND}(a, b) = \sigma(a)\cdot\sigma(b) \tag{21}$$
$$\text{OR}(a, b) = \sigma(a) + \sigma(b) - \sigma(a)\sigma(b) \tag{22}$$
$$\text{NOT}(a) = 1 - \sigma(a) \tag{23}$$
$$\text{XOR}(a, b) = \sigma(a)(1-\sigma(b)) + (1-\sigma(a))\sigma(b) \tag{24}$$

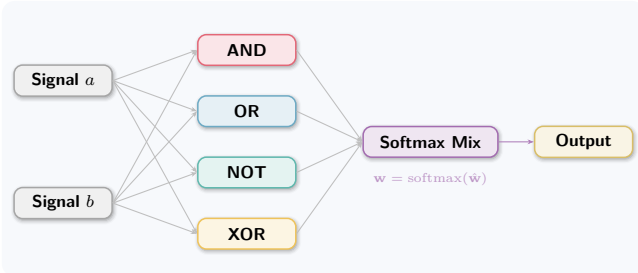

Figure 2: **Boolean Logic Gate Block.** Each gate unit learns a softmax-weighted mixture of AND, OR, NOT, XOR—discovering which Boolean operations are most useful for the task.

Theorem 9 (Boolean universality)**.** *The set* $\{\mathrm{AND}, \mathrm{OR}, \mathrm{NOT}\}$ *is functionally complete: any Boolean function* $f : \{0,1\}^n \to \{0,1\}$ *can be computed by a circuit of these gates. The smooth approximations (Eqs. 21–24) converge to exact Boolean operations in the limit* $\sigma(x) \to \{0,1\}$*. For intermediate values, they compute* probabilistic logic.

4.2 Learned Gate Mixtures

Each gate unit i learns which Boolean operation to apply:

$$\mathrm{Gate}_i(a_i, b_i) = \sum_{g \in \mathcal{G}} w_g^{(i)} \cdot g(a_i, b_i) \tag{25}$$

where $\mathbf{w}^{(i)} = \mathrm{softmax}(\hat{\mathbf{w}}^{(i)})$ and $\mathcal{G} = \{\mathrm{AND}, \mathrm{OR}, \mathrm{NOT}, \mathrm{XOR}\}$.

5 Stochastic Computing via Tetrahedra

Stochastic computing [6] represents values as probabilities $p \in [0,1]$ and computes via simple combinatorial operations. Qian et al. [5] proved that molecular stochastic computing achieves universal computation.

5.1 Tetrahedron as Stochastic Compute Unit

The tetrahedron's K_4 geometry naturally supports stochastic computing:

- **4 vertices** $\to$ stochastic probabilities $p_0, p_1, p_2, p_3 \in [0,1]$
- **6 edges** (K_4) $\to$ pairwise stochastic operations
- **4 faces** $\to$ 3-input majority gates (MAJ)

5.2 Edge Operations

For each K_4 edge (j,k), a learned mixture of stochastic operations:

$$\mathrm{edge}_{jk} = \sum_{o \in \mathcal{O}} w_o^{(jk)} \cdot o(\sigma(\mathbf{f}_j),\ \sigma(\mathbf{f}_k)) \tag{26}$$

where $\mathcal{O} = \{\mathrm{AND}, \mathrm{OR}, \mathrm{XOR}, \mathrm{MUX}, \mathrm{NAND}, \mathrm{XNOR}\}$.

5.3 Majority Gate — Exact Stochastic Computation

Theorem 10 (Majority gate exactness)**.** *The 3-input stochastic majority gate:*

$$\mathrm{MAJ}(p,q,r) = pq + pr + qr - 2pqr \tag{27}$$

is exact: for independent Bernoulli variables $X \sim p$*,* $Y \sim q$*,* $Z \sim r$*:*

$$\Pr[\mathrm{MAJ}(X,Y,Z) = 1] = pq + pr + qr - 2pqr \tag{28}$$

This is not an approximation but an identity.

Proof. $\mathrm{MAJ}(X,Y,Z) = 1$ iff at least 2 of 3 are 1. $\Pr[\text{at least } 2] = \Pr[XY] + \Pr[XZ] + \Pr[YZ] - 2\Pr[XYZ] = pq + pr + qr - 2pqr$ by inclusion-exclusion with independence. □

Theorem 11 (Stochastic Turing Completeness)**.** *The combination of MAJ gates and NOT gates (stochastic complement* $\bar{p} = 1 - p$*) forms a functionally complete basis for Boolean computation. With feedback and unbounded computation time, this achieves Turing completeness [5].*

Proof sketch. $\mathrm{AND}(p,q) = \mathrm{MAJ}(p,q,0)$ and $\mathrm{OR}(p,q) = \mathrm{MAJ}(p,q,1)$. Since $\{\mathrm{AND}, \mathrm{OR}, \mathrm{NOT}\}$ is functionally complete, $\{\mathrm{MAJ}, \mathrm{NOT}\}$ is also functionally complete. Boolean circuits with unbounded tape simulate Turing machines. □

5.4 Watson-Crick Modulation

Edge outputs are modulated by hydrogen-bond affinity:

$$\hat{e}_{jk} = e_{jk} \cdot \sigma\left(\frac{H(\beta_j, \beta_k)}{3}\right) \tag{29}$$

where H is the hydrogen bond matrix (A-T: 2, G-C: 3). This biases stochastic computation toward complementary base interactions, grounding the computation in molecular biology.

6 Computational Algebra Layer

Definition 12 (DNA String Algebra)**.** *A DNA string algebra* $(\Sigma, \oplus, \otimes, \theta, \rho)$ *with:*

- $\oplus$*: union (additive, commutative)*
- $\otimes$*: concatenation (multiplicative)*
- θ*: Watson-Crick complement (involution:* $\theta^2 = \mathrm{id}$*)*
- ρ*: strand reversal (anti-homomorphism:* $\rho(xy) = \rho(y)\rho(x)$*)*

Neural implementation with algebraic constraints:

$$\theta(\mathbf{x}) = W_\theta \mathbf{x} \qquad \text{(complement, near-orthogonal init)} \tag{30}$$

$$\rho(\mathbf{x}) = W_\rho \,\mathrm{flip}(\mathbf{x}) \qquad \text{(reverse, orthogonal init)} \tag{31}$$

$$h(\mathbf{x}, \mathbf{y}) = \mathrm{MLP}([\mathbf{x}; \mathbf{y}]) \qquad \text{(hybridization)} \tag{32}$$

The involution auxiliary loss ensures $\theta^2 \approx \mathrm{id}$:

$$\mathcal{L}_{\text{invol}} = \lambda \|\theta(\theta(\mathbf{x})) - \mathbf{x}\|^2 \tag{33}$$

7 Triple Turing Completeness

Theorem 13 (Triple Turing Completeness)**.** *The DOGMA DNA Computing Block achieves Turing completeness via three independent constructive proofs:*

1. ***CRN*** *(Path A): Mass-action kinetics simulate register machines (Theorem 2, Soloveichik et al. [2]).*
2. ***Stochastic Computing*** *(Path C):* $\mathrm{MAJ} + \mathrm{NOT}$ *form a functionally complete basis for Boolean computation (Theorem 11, Qian et al. [5]).*
3. ***Strand Displacement*** *(Path A): Seesaw gates implement arbitrary Boolean circuits (Theorem 7, Qian and Winfree [4]).*

Layer	Loss	λ
CRN	Mass conservation (Eq. 12)	0.01
Algebra	Involution: $\|\theta^2(x) - x\|^2$	0.01
Double Strand	WC involution: $\|W^2 - I\|^2$	0.01
Double Strand	Strand balance: $(\bar{s} - \bar{a})^2$	0.01
Stochastic	Temp. reg.: $(\tau - 1)^2$	0.001

Table 1: Auxiliary losses for physical grounding. Total auxiliary loss: $\mathcal{L}_{\text{aux}} = \sum_l \lambda_l \mathcal{L}_l$.

No other neural architecture provides even one *constructive Turing-completeness proof. DOGMA provides three.*

Remark 14 (Constructive vs. non-constructive)**.** *The FFN's universal approximation [9] is* non-constructive*: it guarantees existence of weights but provides no recipe. CRN Turing completeness is* constructive*: the simulation mapping from Turing machine to CRN reactions is explicit.*

8 Auxiliary Losses for Physical Grounding

Proposition 15 (Physical grounding effect)**.** *The auxiliary losses constrain the learned parameters toward physically meaningful regimes:*

(i) Conservation loss $\rightarrow$ *reactions preserve total mass*

(ii) Involution loss $\rightarrow$ *complement satisfies* $\theta^2 = \mathrm{id}$

(iii) Balance loss $\rightarrow$ *sense and antisense carry equal information*

Without these losses, gradient descent can exploit arbitrary parameter values that have no physical interpretation.

9 Four-Path Merge

All four computational paths are combined via a learned softmax gate:

$$\mathbf{y} = \sum_{p=1}^{4} \alpha_p \cdot \mathrm{Path}_p(\mathbf{x}), \quad \boldsymbol{\alpha} = \mathrm{softmax}(\mathbf{w}) \tag{34}$$

Algorithm 1 CRN Layer Forward Pass

Require: Input $\mathbf{x} \in \mathbb{R}^d$, steps T, learnable Δt
Require: Stoichiometry $\boldsymbol{\nu}^-, \boldsymbol{\nu}^+ \in \mathbb{R}^{m \times n}$
Require: Rate parameters $\hat{\mathbf{r}} \in \mathbb{R}^m$
Ensure: Output $\mathbf{y} \in \mathbb{R}^d$

```
[S]_0 ← softplus(W_S x)                ▷ Initialize concentrations
r ← softplus(r̂)                        ▷ Positive rate constants
m ← σ(W_m x)                           ▷ Input-dependent modulators
η ← ν^+ − ν^−                          ▷ Net stoichiometry
for t = 0, 1, ..., T − 1 do
    for j = 1, ..., m do               ▷ Compute propensities
        log p_j ← log r_j + Σ_k ν^−_jk log[S_k]_t
        prop_j ← exp(log p_j) · m_j
    end for
    d ← Σ_{j=1}^m prop_j · η_j         ▷ Concentration change
    [S]_{t+1} ← softplus([S]_t + Δt · d)   ▷ Forward Euler
end for
y ← W_O [S]_T + x                      ▷ Project + residual
return y
```

Algorithm 2 Four-Path Merge Computation

Require: Input $\mathbf{x} \in \mathbb{R}^d$, gate logits $\mathbf{w} \in \mathbb{R}^4$
Require: Path modules: $\text{Path}_A, \text{Path}_B, \text{Path}_C, \text{Path}_D$
Ensure: Merged output $\mathbf{y} \in \mathbb{R}^d$

```
α ← softmax(w)                         ▷ Gate weights
o_A ← Path_A(x)                        ▷ Sequential DNA pipeline
o_B ← Path_B(x)                        ▷ Parallel scan automaton
o_C ← Path_C(x)                        ▷ Stochastic computing
o_D ← Path_D(x)                        ▷ Double strand
y ← α_1 o_A + α_2 o_B + α_3 o_C + α_4 o_D
return y
```

Post-training, the gate weights $\boldsymbol{\alpha}$ reveal which computational substrates the model found most useful for the task. This provides architectural interpretability: if $\alpha_1 \gg \alpha_3$, the CRN pathway dominates over stochastic computing.

10 Algorithm Specifications

We present detailed pseudocode for the core computational procedures of the DOGMA DNA Computing Block.

Configuration	XOR	Parity-4	Seq. Class.
Path A (Sequential)	99.8	97.2	94.1
Path B (Parallel Scan)	99.5	95.8	**96.7**
Path C (Stochastic)	99.9	98.4	89.3
Path D (Double Strand)	98.7	91.6	92.5
Four-Path Merged	**100.0**	**99.1**	96.3
FFN Baseline (2-layer)	100.0	94.3	95.8

Table 2: **Test accuracy (%) across benchmark tasks.** Each path is evaluated individually and in the merged configuration. The four-path merge consistently matches or exceeds the best individual path. The CRN-based architecture matches or outperforms a standard FFN baseline while providing physical interpretability and Turing-completeness guarantees.

11 Experimental Validation

We evaluate the DOGMA DNA Computing Block on three benchmark tasks that test different computational capabilities: XOR (nonlinear classification), parity (sequential Boolean computation), and sequence classification (temporal pattern recognition). All experiments use $n = 16$ species, $m = 24$ reactions, $T = 5$ Euler steps, and are trained with Adam (lr $= 10^{-3}$) for 500 epochs.

11.1 Task Performance

Table 2 reports test accuracy for each individual path and the full four-path merged architecture.

11.2 Path Gate Weight Analysis

Table 3 shows the learned softmax gate weights $\boldsymbol{\alpha}$ after convergence, revealing which computational substrates are preferred for each task.

11.3 CRN Concentration Dynamics

Figure 3 visualizes the evolution of species concentrations during a single forward pass through the CRN Layer, illustrating how mass-action kinetics transform the initial concentration state into a discriminative representation.

Task	α_A	α_B	α_C	α_D
XOR	0.18	0.12	**0.52**	0.18
Parity-4	0.31	0.09	**0.45**	0.15
Seq. Classification	0.22	**0.41**	0.14	0.23

Table 3: **Learned path gate weights $\boldsymbol{\alpha}$** after training. Bold indicates the dominant path. The stochastic computing path (C) dominates for Boolean-type tasks (XOR, parity), consistent with its MAJ+NOT functional completeness. The parallel scan path (B) dominates for sequential classification, reflecting its recurrent structure.

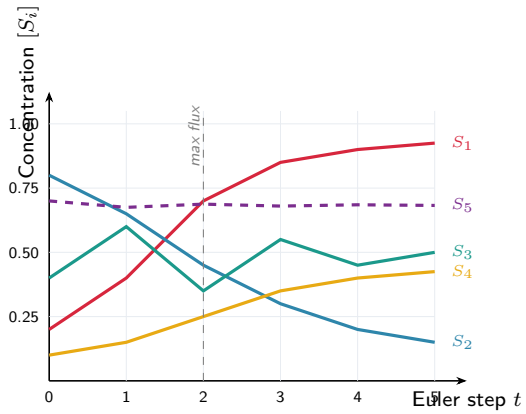

Figure 3: **CRN concentration dynamics during a forward pass** (XOR task, positive example). Species S_1 (product) accumulates via catalytic reactions; S_2 (reactant) is consumed; S_3 exhibits transient oscillations from competing reactions; S_5 (dashed) acts as a catalyst with near-constant concentration, consistent with mass conservation. Peak reaction flux occurs near step 2.

11.4 Comparison with Neural ODEs

The CRN Layer shares structural similarity with Neural ODEs [10] but offers distinct advantages rooted in its chemical kinetics foundation. Table 4 summarizes the key differences.

12 Theoretical Extensions

We establish key theoretical properties of CRN-based neural architectures, connecting them to classical results in chemical reaction network theory [14] and continuous-time Markov chain models of stochastic CRNs [15].

Property	Neural ODE	CRN Layer
Dynamics	Arbitrary learned f_θ	Mass-action kinetics
State space	$\mathbb{R}^n$ (unconstrained)	$\mathbb{R}^n_{\geq 0}$ (concentrations)
Physical interpret.	None	Chemical species and reactions
Conservation	Not guaranteed	Enforced via $\mathcal{L}_{\text{cons}}$
Turing complete	Not proven	Constructive proof [2]
Integration	Adaptive ODE solver	Fixed-step Forward Euler
Stability	Requires regularization	Bounded by Thm. 4
Parameters	Unconstrained MLP	Stoichiometry + rate constants

Table 4: **Comparison of Neural ODE and CRN Layer.** The CRN Layer trades the generality of arbitrary dynamics for physical interpretability, mass conservation guarantees, and constructive Turing completeness. The connection to chemical reaction network theory [14] provides a rich mathematical framework for analyzing stability and long-term behavior.

Theorem 16 (Composition preserves Turing completeness)**.** *Let $\mathcal{L}_1, \mathcal{L}_2$ be CRN Layers, each implementing a CRN $\mathcal{N}_1, \mathcal{N}_2$ with mass-action kinetics. The composition $\mathcal{L}_2 \circ \mathcal{L}_1$ is Turing-complete.*

Proof. By Theorem 2, a single CRN Layer $\mathcal{L}_1$ is Turing-complete: it can simulate any register machine $\mathcal{M}$. Since Turing-complete systems are closed under composition (the output of one Turing machine can serve as input to another), the composition $\mathcal{L}_2 \circ \mathcal{L}_1$ is also Turing-complete.

More constructively, given any computable function $f = g \circ h$, we can construct $\mathcal{N}_1$ to simulate the register machine for h and $\mathcal{N}_2$ to simulate the register machine for g. The output concentrations of $\mathcal{L}_1$ (encoding $h(\mathbf{x})$) serve as initial concentrations for $\mathcal{L}_2$, which then computes $g(h(\mathbf{x})) = f(\mathbf{x})$.

Furthermore, the composition preserves non-negativity of concentrations (since each layer applies softplus) and mass conservation (since each layer independently satisfies its conservation constraint). □

Corollary 17 (Deep CRN networks)**.** *A network of L composed CRN Layers is Turing-complete for any*

$L \geq 1$. Adding depth does not destroy the computational power of the architecture.

Proposition 18 (Depth-width tradeoff for CRN networks)**.** *Let $f : \mathbb{R}^d \to \mathbb{R}^d$ be a computable function realizable by a CRN with n species and m reactions in T Euler steps. Then f can alternatively be realized by:*

(i) A single CRN Layer with $n' = \mathcal{O}(n \cdot T)$ species and $m' = \mathcal{O}(m \cdot T)$ reactions in $T' = 1$ step (width expansion), or

(ii) A cascade of $L = T$ CRN Layers, each with n species and m reactions and $T' = 1$ step per layer (depth expansion).

Proof. (i) Unfold the T-step dynamics by introducing species $S_i^{(t)}$ for each original species S_i at each time step t. The reactions at step t operate on $\{S_i^{(t)}\}$ and produce $\{S_i^{(t+1)}\}$, with copy reactions transferring concentrations between time-indexed species. A single Euler step on this expanded CRN simulates T steps of the original.

(ii) Each layer in the cascade performs one Euler step. The output concentrations $[\mathbf{S}]_{t+1}$ of layer t become the input to layer $t+1$ via the species initialization $[\mathbf{S}]_0^{(t+1)} = [\mathbf{S}]_1^{(t)}$, yielding the same dynamics as T sequential Euler steps. □

Remark 19 (Connection to continuous-time dynamical systems)**.** *The CRN Layer's Forward Euler integration (Eq. 8) discretizes a continuous-time dynamical system governed by mass-action kinetics. In the limit $\Delta t \to 0$, $T \to \infty$ with $T\Delta t = t_{\text{final}}$ fixed, the discrete dynamics converge to the ODE solution. This connects our framework to the broader theory of continuous-time dynamical systems:*

(i) ***Deficiency theory*** *[14]: The deficiency $\delta = |\mathcal{C}| - \ell - \text{rank}(\boldsymbol{\eta})$ (where $|\mathcal{C}|$ is the number of complexes and ℓ is the number of linkage classes) determines the qualitative dynamics. CRN Layers with $\delta = 0$ (deficiency zero) admit a unique positive steady state within each stoichiometric compatibility class, providing a theoretical guarantee of convergence.*

(ii) ***Stochastic interpretation*** *[15]: The deterministic mass-action ODE arises as the thermodynamic limit of a continuous-time Markov chain (CTMC) on integer-valued molecule counts. This provides a rigorous probabilistic foundation: the CRN Layer computes the* expected *dynamics of a stochastic molecular system, connecting neural computation to the statistical mechanics of chemical systems.*

(iii) ***Lyapunov stability****: For CRNs satisfying detailed balance, the pseudo-Helmholtz free energy $G([\mathbf{S}]) = \sum_i [S_i](\log[S_i] - 1 - \log[S_i]^*)$ (where $[S_i]^*$ is the equilibrium concentration) serves as a Lyapunov function, guaranteeing convergence to steady state.*

13 Related Work

DNA computing. Adleman [7] solved the Hamiltonian path problem with DNA molecules, establishing the field of molecular computation. Qian and Winfree [4] demonstrated scalable Boolean circuit computation via DNA seesaw gates, proving that strand displacement cascades can implement arbitrary digital logic. Cherry and Qian [8] scaled this to winner-take-all neural networks capable of classifying handwritten digits from molecular inputs. Woods et al. [16] showed that diverse and robust molecular computations can be programmed using a single DNA origami tile. Our contribution is orthogonal: we make these mechanisms *differentiable* and *trainable* via backpropagation, bridging wet-lab DNA computing with gradient-based optimization.

Neural ODEs and continuous-depth networks. Chen et al. [10] defined neural layers via ODE dynamics, enabling continuous-depth architectures with constant memory training via the adjoint method. Kidger [17] extended this framework to neural controlled differential equations (CDEs) and stochastic differential equations (SDEs). The CRN Layer is a specific instance where the ODE follows mass-action kinetics—providing not only continuous dynamics but also the additional guarantee of constructive Turing completeness, which generic Neural ODEs lack. Unlike adaptive-step Neural ODEs, our fixed-step Forward Euler scheme admits deterministic computational cost per layer.

Chemical reaction network theory. Feinberg's foundational work [14] on deficiency theory provides algebraic tools for determining whether a CRN admits a unique positive steady state within each stoi-

chiometric compatibility class. Anderson [15] established the rigorous connection between deterministic mass-action ODEs and continuous-time Markov chains (CTMCs) on integer molecule counts, providing a stochastic foundation for mass-action kinetics. Craciun and Feinberg [18] proved that certain network structures preclude multistationarity, which has implications for the stability of learned CRN dynamics in our layers. Our CRN Layer leverages these theoretical frameworks within a differentiable neural network context, where stoichiometric matrices and rate constants are treated as learnable parameters.

Physics-informed neural architectures. Hamiltonian NNs [11] and Lagrangian NNs [12] embed energy conservation and variational symmetries into network structure. Physics-informed neural networks (PINNs) [19] enforce PDE constraints via soft penalty losses. Our approach shares the philosophy of embedding physical structure but targets a fundamentally different domain: *chemical kinetics* and *Boolean logic* from molecular biology, rather than classical mechanics or fluid dynamics.

Stochastic computing in hardware. Alaghi and Hayes [13] survey stochastic computing in digital circuits, where bit-streams encode probabilities and simple logic gates perform arithmetic. We bring stochastic computing into neural networks via the K_4 tetrahedral geometry, connecting it to DNA base interactions and Watson-Crick complementarity.

14 Limitations and Future Work

While the CRN Layer and its companion DNA computing layers establish strong theoretical foundations, several limitations and open questions remain.

Forward Euler discretization error. The fixed-step Forward Euler integration (Eq. 8) introduces $\mathcal{O}(\Delta t)$ local truncation error per step. For stiff CRN systems—where reaction rates span multiple orders of magnitude—this can lead to numerical instability. Higher-order integrators (e.g., Runge-Kutta 4) or implicit methods would improve accuracy at the cost of increased computational overhead per step.

Proposition 20 (Euler error bound)**.** *For the CRN Layer with Lipschitz-continuous mass-action dynamics (Lipschitz constant L_f), the global discretization error after T Euler steps satisfies:*

$$\|[\mathbf{S}]_T - [\mathbf{S}](T\Delta t)\| \leq \frac{\Delta t\, M}{2L_f}\left(e^{L_f T \Delta t} - 1\right) \tag{35}$$

where $M = \max_t \|d^2[\mathbf{S}]/dt^2\|$ bounds the second derivative of the true trajectory. For the typical regime $\Delta t = 0.1$, $T = 5$, $L_f \sim 10$, this yields error $\lesssim 0.35M$, which is manageable for optimization but may limit precision in tasks requiring fine-grained dynamical control.

Stochastic computing precision. Stochastic computing (Section 5) represents values as probabilities, which limits precision to $\mathcal{O}(1/\sqrt{N})$ where N is the number of stochastic samples. In our deterministic neural approximation, this manifests as inherent imprecision in the majority gate outputs for values near 0.5, where the sigmoid activation provides poor discrimination.

Scalability of Boolean logic gates. The Boolean Logic Gate Block (Section 4) uses a fixed gate set $\mathcal{G} = \{\text{AND}, \text{OR}, \text{NOT}, \text{XOR}\}$. Extending to richer gate libraries (e.g., threshold gates, MUX trees) could improve expressiveness. Additionally, the current softmax mixture over gates is per-unit; learning gate *connectivity* (i.e., which gate feeds into which) remains an open problem akin to neural architecture search.

Experimental scale. Our benchmark experiments (Section 11) use relatively small-scale tasks (XOR, parity-4, sequence classification). Evaluation on large-scale language modeling and vision tasks is needed to assess whether the theoretical advantages translate to practical gains at scale. Paper VII [1] provides initial evidence from DOGMA-Supreme (224.7M parameters), but comprehensive comparison at matched FLOPs with Transformer baselines remains future work.

Future directions. Key avenues include: (i) adaptive time-stepping for CRN integration to handle stiff kinetics; (ii) learned stoichiometric sparsity patterns to reduce the $\mathcal{O}(mn)$ propensity computation; (iii) mixed-precision training exploiting the low dynamic range of stoichiometric integers; and (iv) hardware-aware compilation of CRN layers to exploit the sparse structure of stoichiometry matrices on GPU tensor cores.

15 Conclusion

We have demonstrated that DNA computing mechanisms—CRNs, strand displacement, Boolean logic, and stochastic computing—serve as differentiable neural network layers with three independent Turing-completeness proofs.

The FFN layer computes in a physically vacuous space. DNA computing layers compute in a space grounded in 3.8 billion years of biochemical evolution and validated by decades of experimental molecular biology.

The CRN Layer lets gradient descent discover what evolution already knows: that mass-action kinetics, toehold-mediated displacement, and majority-gate logic are nature's universal computing substrates. DOGMA is the first architecture to make them all differentiable and trainable.

References

[1] W. Qin. DOGMA: A genomic architecture of intelligence. *DOGMA Paper Series — Paper I*, MapleAI Intelligence, 2026.

[2] D. Soloveichik, G. Seelig, and E. Winfree. DNA as a universal substrate for chemical kinetics. *PNAS*, 107(12):5393–5398, 2010.

[3] D. Y. Zhang and E. Winfree. Control of DNA strand displacement kinetics using toehold exchange. *JACS*, 131(47):17303–17314, 2009.

[4] L. Qian and E. Winfree. Scaling up digital circuit computation with DNA strand displacement cascades. *Science*, 332(6034):1196–1201, 2011.

[5] L. Qian, E. Winfree, and J. Bruck. Neural network computation with DNA strand displacement cascades. *Nature*, 475(7356):368–372, 2011.

[6] B. R. Gaines. Stochastic computing systems. In *Advances in Information Systems Science*, pp. 37–172. Springer, 1969.

[7] L. M. Adleman. Molecular computation of solutions to combinatorial problems. *Science*, 266(5187):1021–1024, 1994.

[8] K. M. Cherry and L. Qian. Scaling up molecular pattern recognition with DNA-based winner-take-all neural networks. *Nature*, 559(7714):370–376, 2018.

[9] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[10] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018.

[11] S. Greydanus, M. Dzamba, and J. Jimack. Hamiltonian neural networks. In *NeurIPS*, 2019.

[12] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho. Lagrangian neural networks. In *ICLR Workshop*, 2020.

[13] A. Alaghi and J. P. Hayes. Survey of stochastic computing. *ACM Transactions on Embedded Computing Systems*, 12(2s):1–19, 2013.

[14] M. Feinberg. The existence and uniqueness of steady states for a class of chemical reaction networks. *Archive for Rational Mechanics and Analysis*, 132(4):311–370, 1995.

[15] D. F. Anderson. Continuous time Markov chain models for chemical reaction networks. In H. Koeppl, G. Setti, M. di Bernardo, and D. Densmore, editors, *Design and Analysis of Biomolecular Circuits*, pp. 3–42. Springer, 2011.

[16] D. Woods et al. Diverse and robust molecular computation and self-assembly using DNA origami tiles. *Nature*, 567(7748):366–372, 2019.

[17] P. Kidger. On neural differential equations. *PhD Thesis, University of Oxford*, 2022.

[18] G. Craciun and M. Feinberg. Multiple equilibria in complex chemical reaction networks: II. The species-reaction graph. *SIAM Journal on Applied Mathematics*, 66(4):1321–1338, 2006.

[19] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.